02 · Citizen dashboard

The citizen's home page: latest values against their usual, a trend chart, alerts, the next visit, and how well the app knows them.

New to the syntax? Read **00 · How to read this code** first; this guide explains what is specific to this feature.

What this feature does

The dashboard greets the citizen and shows: the newest value of each vital (out-of-range ones first, drawn larger), a 30-day chart of one vital with the normal range shaded and flagged readings as large red dots, unread alerts, the next booked visit, and a "How well we know your normal" card with a progress bar per vital.

Files involved

Only the files this feature uses.

```
backend/app/
├── api/routes/vitals.py       GET /health/summary (everything for the page), GET /health/baseline
└── services/readings.py       baseline_of(): the person's usual value for display
frontend/src/
├── pages/citizen/Dashboard.tsx          the page
├── components/app/TrendChart.tsx        the line chart (Recharts library)
├── components/app/ModelInsights.tsx     BaselineProgress: one bar per vital
└── services/health.ts                   getSummary(), getMetrics(), getBaselineStatus()
```

How the pieces connect

1. The page asks for three things at once: the summary, the vitals list and the baseline status.
2. `GET /health/summary` reads the last 90 days of readings and builds latest values, the 30-day trend, counts, alerts, the next visit and the health centre in one answer.
3. `GET /health/baseline` reports, per vital, how many readings the model has and its stage (learning, blending, personal).
4. The page sorts the latest values, draws the chart for the chosen vital and the progress bars.

Code walkthrough

Every excerpt below is copied straight from the file, with its real line numbers, when this guide was generated.

1. `backend/app/api/routes/vitals.py` · summary (first part)

```
214 async def summary(user: User = Depends(require_onboarding_user), db: AsyncSession = Depends(get_session)) ->
        SummaryOut:
215     now = now_utc()
216     window_start = now - timedelta(days=BASELINE_WINDOW_DAYS)
217     rows = (
218         await db.execute(
219             select(HealthRecord, ProcessedData)
220             .outerjoin(ProcessedData, ProcessedData.health_record_id == HealthRecord.id)
221             .where(HealthRecord.user_id == user.id, HealthRecord.timestamp >= window_start)
222             .order_by(HealthRecord.timestamp.desc())
223             .limit(SUMMARY_READINGS_CAP)
224         )
225     ).all()
226
227     # Newest value of each vital, compared with that vital's earlier values in the window.
228     latest: list[LatestVital] = []
229     for key in VITALS:
230         series = [(record, record.vitals[key]) for record, _ in rows if record.vitals.get(key) is not None]
231         if not series:
232             continue
233         newest_record, value = series[0]
234         base = baseline_of(key, [v for _, v in reversed(series[1:])])
235         latest.append(
```

```
            LatestVital(
                key=key,
                value=value,
                measured_at=newest_record.timestamp,
                position=position(key, value),
                baseline=BaselineOut(usual=round(base.usual, 2), spread=round(base.spread, 2), n=base.n) if
                    base else None,
            )
        )

    trend_start = now - timedelta(days=TREND_WINDOW_DAYS)
    trend = [
        TrendPoint(measured_at=record.timestamp, vitals=record.vitals, flagged=bool(processed and processed.
            is_anomaly))
        for record, processed in reversed(rows)
        if record.timestamp >= trend_start
    ]

```

What the code means

- `select(HealthRecord, ProcessedData).outerjoin(...)` loads each reading together with its score; `outerjoin` keeps readings that have no score row.
- `.where(..., HealthRecord.timestamp >= window_start)` limits it to the last 90 days; `.limit(500)` caps the work.
- `for key in VITALS:` goes through every vital. `series` is that vital's values, newest first; `series[0]` is the newest.
- `baseline_of(key, [v for _, v in reversed(series[1:])])` builds the usual value from the **earlier** readings, oldest first (the _ ignores the record part of each pair).
- `position(key, value)` says below, normal or above the published range.

2. `backend/app/services/readings.py` · baseline_of

```
def baseline_of(key: str, values: list[float]) -> Baseline | None:
    """The scorer's own baseline for display, or None below MIN_BASELINE_READINGS:
    no baseline is better than a fake one. `values` oldest first."""
    if len(values) < MIN_BASELINE_READINGS:
        return None
    usual, spread = robust_baseline(key, values)
    return Baseline(usual=usual, spread=spread, n=min(len(values), BASELINE_WINDOW))
```

What the code means

- Below 7 readings it returns `None`: no baseline is better than a fake one, and the page then shows the published range instead.
- `robust_baseline` is the model's own median and spread, so the dashboard shows exactly what the model compares against.

3. `backend/app/api/routes/vitals.py` · baseline_status

```
@router.get("/baseline", response_model=BaselineStatusOut)
async def baseline_status(
    user: User = Depends(require_onboarding_user), db: AsyncSession = Depends(get_session)
) -> BaselineStatusOut:
    """Per vital: how many readings the model has, and how far it is from published ranges to personal."""
    history = await earlier_vitals(db, user.id, now_utc() + timedelta(minutes=1))
    vitals = []
    for key in VITALS:
        values = [float(h[key]) for h in history if h.get(key) is not None]
        if not values:
            continue
        median, spread = robust_baseline(key, values) if len(values) >= MIN_BASELINE_READINGS else (None, No
            ne)
        vitals.append(
            VitalBaseline(
                key=key,
                readings=len(values),
                stage=stage_of(len(values)),
                weight=round(baseline_weight(len(values)), 3),
                baseline=round(median, 2) if median is not None else None,
                spread=round(spread, 2) if spread is not None else None,
                readings_to_start=max(0, MIN_BASELINE_READINGS - len(values)),
```

```
                readings_to_full=max(0, FULL_BASELINE_READINGS - len(values)),
            )
        )
    return BaselineStatusOut(
        model_version=MODEL_VERSION,
        min_readings=MIN_BASELINE_READINGS,
        full_readings=FULL_BASELINE_READINGS,
        window=min(BASELINE_WINDOW, SCORING_HISTORY),
        vitals=vitals,
    )
```

What the code means

- `now_utc() + timedelta(minutes=1)` includes every reading up to now.
- For each vital: `stage_of(n)` gives learning / blending / personal, and `baseline_weight(n)` how much the personal baseline counts (0 to 1).
- `max(0, MIN_BASELINE_READINGS - n)` is "readings still needed", never negative.

4. `frontend/src/pages/citizen/Dashboard.tsx` · CitizenDashboard (data loading)

```
export function CitizenDashboard() {
  const { user } = useAuth();
  const { data, error, loading, reload } = useResource(
    (signal) => Promise.all([getSummary(signal), getMetrics(), getBaselineStatus(signal)]),
    "summary",
  );
  const [chosen, setChosen] = useState<string | null>(null);

  const [summary, info, baselines] = data ?? [null, null, null];
  const metrics = useMemo(() => new Map((info?.metrics ?? []).map((m) => [m.key, m])), [info]);
  const charted = useMemo(
    () => (info?.metrics ?? []).filter((m) => summary?.trend.some((p) => p.vitals[m.key] !== undefined)),
    [info, summary],
  );
  const metric: Metric | undefined = charted.find((m) => m.key === chosen) ?? charted[0];

  if (loading && !data) return <Loading label="Loading your dashboard" />;
  if (error || !summary || !info) return <ErrorNote message={error ?? "Could not load your dashboard."} onRe
      try={reload} />;

  const unreviewed = !summary.facility || summary.facility.workers_assigned === 0;
  const offNormal = summary.latest.filter((v) => v.position !== "normal");
  const inRange = summary.latest.filter((v) => v.position === "normal");
  const alerts = groupAlerts(summary.alerts.map((a) => a.message));
  const points = metric
    ? summary.trend
        .filter((p) => p.vitals[metric.key] !== undefined)
        .map((p) => ({ measured_at: p.measured_at, value: p.vitals[metric.key], flagged: p.flagged }))
    : [];

  return (
```

What the code means

- `useResource((signal) => Promise.all([...]), "summary")` loads three requests in parallel; `signal` lets the request be cancelled if the user leaves the page.
- `const [summary, info, baselines] = data ?? [null, null, null]` unpacks the three answers (or nulls while loading).
- `useMemo(() => new Map(...), [info])` builds a lookup from vital key to its details once, not on every re-draw.
- `if (loading && !data) return <Loading/>` and the error line are **early returns**: the rest of the page only runs with data.
- `offNormal` / `inRange` split the latest values so out-of-range ones are drawn first and larger.

5. `frontend/src/pages/citizen/Dashboard.tsx` · groupAlerts

Several alerts often end with the same sentence; this lifts the shared ending out so it is shown once.

```
function groupAlerts(messages: string[]): { lines: string[]; shared: string | null } {
  const split = (m: string) => m.split(/(?<=\.)\s+/);
  if (messages.length < 2) return { lines: messages, shared: null };
  const parts = messages.map(split);
  const shared: string[] = [];
  for (let i = 1; i <= Math.min(...parts.map((p) => p.length)) - 1; i += 1) {
    const tail = parts[0][parts[0].length - i];
    if (parts.every((p) => p[p.length - i] === tail)) shared.unshift(tail);
    else break;
  }
  if (shared.length === 0) return { lines: messages, shared: null };
  return {
    lines: parts.map((p) => p.slice(0, p.length - shared.length).join(" ")),
    shared: shared.join(" "),
  };
}
```

What the code means

- `m.split(/(?<=\.)\s+/)` splits a message into sentences: the regular expression matches spaces that come right after a full stop.
- The loop compares sentences from the end of every message; while all messages share the same last sentence, it is moved into `shared`.
- `unshift` adds to the front of a list; `slice(0, n)` keeps the first n items.

6. `frontend/src/components/app/TrendChart.tsx` · FlagDot

```
function FlagDot(props: DotProps & { payload?: TrendPoint }) {
  const { cx, cy, payload } = props;
  if (cx === undefined || cy === undefined || !payload) return null;
  return payload.flagged ? (
    <circle cx={cx} cy={cy} r={5} fill="rgb(var(--hn-sev-critical))" stroke="rgb(var(--hn-surface))" strokeW
        idth={2} />
  ) : (
    <circle cx={cx} cy={cy} r={3} fill="rgb(var(--hn-brand))" stroke="rgb(var(--hn-surface))" strokeWidth={2
        } />
  );
}
```

What the code means

- A custom dot for the chart: Recharts passes the position (`cx`, `cy`) and the data point (`payload`).
- Flagged readings get a bigger red circle (`r={5}`), normal ones a small brand-coloured one, so colour is never the only signal.

7. `frontend/src/components/app/TrendChart.tsx` · TrendChart

```
export function TrendChart({ metric, points, height = 240 }: { metric: Metric; points: TrendPoint[]; height?
    : number }) {
  const values = points.map((p) => p.value);
  const lows = [...values, ...(metric.normal ? [metric.normal[0]] : [])];
  const highs = [...values, ...(metric.normal ? [metric.normal[1]] : [])];
  const pad = (Math.max(...highs) - Math.min(...lows)) * 0.15 || 1;
  const domain: [number, number] = [
    Math.max(metric.plausible[0], Math.floor(Math.min(...lows) - pad)),
    Math.min(metric.plausible[1], Math.ceil(Math.max(...highs) + pad)),
  ];

  return (
    <figure>
      <div style={{ height }}>
        <ResponsiveContainer width="100%" height="100%">
          <LineChart data={points} margin={{ top: 8, right: 12, bottom: 0, left: -8 }}>
            <CartesianGrid stroke={LINE} strokeDasharray="0" vertical={false} />
            {metric.normal && (
              <ReferenceArea y1={metric.normal[0]} y2={metric.normal[1]} fill="rgb(var(--hn-success))" fillO
                  pacity={0.08} strokeOpacity={0} />
            )}
```

```
48              <XAxis
49                dataKey="measured_at"
50                tickFormatter={fmtShortDay}
51                tick={{ fill: INK_SOFT, fontSize: 12 }}
52                axisLine={{ stroke: LINE }}
53                tickLine={false}
54                minTickGap={24}
    ... (37 more lines in the file: the X/Y axes, tooltip and line follow)
```

What the code means

- `{ metric, points, height = 240 }` destructures the props, with 240 as the default height.
- `lows` and `highs` include the normal range, so the shaded band is always visible; `pad` adds 15% space.
- `<ResponsiveContainer>` makes the chart fill its box; `<ReferenceArea y1 y2>` shades the normal range.
- `{metric.normal && (...)}` draws the band only for vitals that have a normal range.

8. `frontend/src/components/app/ModelInsights.tsx` · BaselineProgress

```
86  export function BaselineProgress({ status, metrics }: { status: BaselineStatus; metrics: Map<string, Metric>
        }) {
87    return (
88      <ul className="flex flex-col gap-4">
89        {status.vitals.map((v) => {
90          const m = metrics.get(v.key);
91          if (!m) return null;
92          const filled = Math.min(100, Math.round((v.readings / status.full_readings) * 100));
93          const usual = v.baseline !== null && v.spread !== null ? `usual ${fmt(m, v.baseline)} ± ${fmt(m, v.s
                pread)} ${m.unit}` : null;
94          let detail: string;
95          if (v.stage === "learning") {
96            detail = `${v.readings_to_start} more reading${v.readings_to_start === 1 ? "" : "s"} until we star
                t learning your normal`;
97          } else if (v.stage === "blending") {
98            detail = `${usual} · ${Math.round(v.weight * 100)}% based on you · ${v.readings_to_full} more to b
                e fully personal`;
99          } else {
100           detail = `${usual} · judged against your own readings`;
101         }
102         return (
103           <li key={v.key}>
104             <div className="mb-1 flex flex-wrap items-baseline justify-between gap-2 text-sm">
105               <span className="font-medium text-ink">{m.label}</span>
106               <span className={cx("text-xs font-semibold", v.stage === "personal" ? "text-success" : "text-i
                    nk-soft")}>{STAGE_TEXT[v.stage]}</span>
107             </div>
108             <div
109               className="h-2 overflow-hidden rounded-full bg-paper-raised"
110               role="progressbar"
111               aria-label={`${m.label}: ${v.readings} of ${status.full_readings} readings`}
112               aria-valuemin={0}
113               aria-valuemax={status.full_readings}
114               aria-valuenow={Math.min(v.readings, status.full_readings)}
115             >
    ... (9 more lines in the file: the bar and its text follow)
```

What the code means

- `Math.min(100, Math.round(v.readings / status.full_readings * 100))` turns readings into a percentage for the bar width.
- The `if / else if / else` builds the sentence for each stage: readings still needed, the share based on the person, or their usual value.
- `role="progressbar"` with `aria-valuenow` makes the bar readable by screen readers.

Syntax used in this feature

Code	What it means
`outerjoin(A, cond)`	SQL LEFT JOIN: keep rows of the first table even without a match.
`reversed(xs)`	The list in the opposite order.
`Promise.all([...])`	Run several requests at once and wait for all.
`useMemo(fn, [deps])`	Remember a computed value until deps change.
`/(?<=\.)\s+/`	A regular expression: spaces that follow a full stop.
`style={{ width: ${n}% }}`	An inline style computed in code.